ARTIFICIAL INTELLIGENCE LABORATORY

EXPERIMENT 1: WATER JUG PROBLEM

Aim

To implement the Water Jug Problem using Breadth First Search (BFS) with dynamic input.

Algorithm

1.

Read the capacities of the two jugs and the target amount.

2.
3.

Represent each state as `(x, y)`, where `x` and `y` are the current amounts of water.

4.
5.

Start from `(0, 0)`.

6.
7.

Generate states by filling, emptying, and transferring water between the jugs.

8.
9.

Use BFS to search for the target state.

10.
11.

Display the solution path.

12.

Program

```
from collections import deque
```

```
def water_jug(cap1, cap2, target):
    start = (0, 0)
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        (x, y), path = queue.popleft()

        if x == target or y == target:
            return path

        states = [
            (cap1, y),
            (x, cap2),
            (0, y),
            (x, 0)
        ]

        # Jug 1 -> Jug 2
        amount = min(x, cap2 - y)
        states.append((x - amount, y + amount))

        # Jug 2 -> Jug 1
        amount = min(y, cap1 - x)
        states.append((x + amount, y - amount))

        for state in states:
            if state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))

    return None


cap1 = int(input("Enter capacity of Jug 1: "))
cap2 = int(input("Enter capacity of Jug 2: "))
target = int(input("Enter target amount: "))

solution = water_jug(cap1, cap2, target)

if solution:
    print("\nSolution:")
    for state in solution:
        print(state)
else:
    print("No solution possible.")
```

Sample Input

```
Enter capacity of Jug 1: 4
Enter capacity of Jug 2: 3
Enter target amount: 2
```

Sample Output

```
Solution:
(0, 0)
(0, 3)
(3, 0)
```

```
(3, 3)
(4, 2)
(0, 2)
```

Result

Thus, the Water Jug Problem was successfully implemented using BFS with dynamic input.

EXPERIMENT 2: MISSIONARIES AND CANNIBALS

Aim

To implement the Missionaries and Cannibals Problem using Breadth First Search (BFS) with dynamic input.

Algorithm

1.

Read the number of missionaries, cannibals, and boat capacity.

2.
3.

Represent a state as `(M, C, B)`.

4.
5.

`M` represents missionaries on the left bank.

6.
7.

`C` represents cannibals on the left bank.

8.
9.

`B` represents the boat position.

10.

11.

Generate all possible valid movements.

12.
13.

Reject states where cannibals outnumber missionaries when missionaries are present.

14.
15.

Use BFS to reach the goal state.

16.
17.

Display the solution.

18.

Program

```
from collections import deque

def is_valid(m, c, total_m, total_c):
    if m < 0 or c < 0 or m > total_m or c > total_c:
        return False

    # Left bank
    if m > 0 and m < c:
        return False

    # Right bank
    mr = total_m - m
    cr = total_c - c

    if mr > 0 and mr < cr:
        return False

    return True


def solve(total_m, total_c, boat_capacity):
    start = (total_m, total_c, 0)
    goal = (0, 0, 1)

    queue = deque([(start, [start])])
    visited = {start}

    moves = []

    for m in range(boat_capacity + 1):
        for c in range(boat_capacity + 1):
```

```
            if 1 <= m + c <= boat_capacity:
                moves.append((m, c))

    while queue:
        (m, c, boat), path = queue.popleft()

        if (m, c, boat) == goal:
            return path

        for dm, dc in moves:

            if boat == 0:
                new_m = m - dm
                new_c = c - dc
                new_boat = 1
            else:
                new_m = m + dm
                new_c = c + dc
                new_boat = 0

            state = (new_m, new_c, new_boat)

            if is_valid(new_m, new_c, total_m, total_c):
                if state not in visited:
                    visited.add(state)
                    queue.append((state, path + [state]))

    return None


m = int(input("Enter number of missionaries: "))
c = int(input("Enter number of cannibals: "))
capacity = int(input("Enter boat capacity: "))

solution = solve(m, c, capacity)

if solution:
    print("\nSolution:")
    for state in solution:
        print(state)
else:
    print("No solution possible.")
```

Sample Input

```
Enter number of missionaries: 3
Enter number of cannibals: 3
Enter boat capacity: 2
```

Sample Output

```
Solution:
(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
```

```
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(0, 2, 0)
(0, 0, 1)
```

Result

Thus, the Missionaries and Cannibals Problem was successfully implemented using BFS with dynamic input.

EXPERIMENT 3: 8 QUEENS PROBLEM

Aim

To implement the N-Queens Problem using the Backtracking technique with dynamic input.

Algorithm

1.

Read the number of queens.

2.
3.

Place one queen in each row.

4.
5.

Check whether the selected position is safe.

6.
7.

A queen must not share the same column or diagonal with another queen.

8.
9.

If the position is safe, place the queen.

10.
11.

Otherwise, backtrack and try another position.

12.
13.

Continue until all queens are placed.

14.
15.

Display the chessboard.

16.

Program

```
def is_safe(board, row, col):
    for i in range(row):

        # Same column
        if board[i] == col:
            return False

        # Same diagonal
        if abs(board[i] - col) == abs(i - row):
            return False

    return True


def solve_n_queens(board, row, n):
    if row == n:
        return True

    for col in range(n):
        if is_safe(board, row, col):
            board[row] = col

            if solve_n_queens(board, row + 1, n):
                return True

            board[row] = -1

    return False


n = int(input("Enter number of queens: "))

board = [-1] * n

if solve_n_queens(board, 0, n):
    print("\nSolution:")

    for row in range(n):
        for col in range(n):
            if board[row] == col:
                print("Q", end=" ")
```

```
            else:
                print(".", end=" ")
        print()
else:
    print("No solution exists.")
```

Sample Input

```
Enter number of queens: 8
```

Sample Output

```
Solution:
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

Result

Thus, the 8-Queens Problem was successfully implemented using the Backtracking technique with dynamic input.

EXPERIMENT 4: 8 PUZZLE PROBLEM

Aim

To implement the 8-Puzzle Problem using the A* Search algorithm with dynamic input.

Algorithm

1.

Read the initial and goal states from the user.

2.
3.

Use `0` to represent the blank space.

4.
5.

Calculate the Manhattan distance as the heuristic.

6.
7.

Generate possible movements of the blank space.

8.
9.

Calculate `f(n) = g(n) + h(n)`.

10.
11.

Select the state with the minimum `f(n)`.

12.
13.

Continue until the goal state is reached.

14.
15.

Display the solution path.

16.

Program

```
import heapq


def heuristic(state, goal):
    distance = 0

    for i, value in enumerate(state):
        if value == 0:
            continue

        goal_index = goal.index(value)

        r1, c1 = divmod(i, 3)
        r2, c2 = divmod(goal_index, 3)

        distance += abs(r1 - r2) + abs(c1 - c2)

    return distance


def get_neighbors(state):
    neighbors = []
```

```
    zero = state.index(0)
    row, col = divmod(zero, 3)

    moves = [
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1)
    ]

    for dr, dc in moves:
        new_row = row + dr
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:

            new_zero = new_row * 3 + new_col

            new_state = list(state)

            new_state[zero], new_state[new_zero] = \
                new_state[new_zero], new_state[zero]

            neighbors.append(tuple(new_state))

    return neighbors


def a_star(start, goal):
    queue = []

    h = heuristic(start, goal)

    heapq.heappush(
        queue,
        (h, 0, start, [start])
    )

    visited = set()

    while queue:
        f, cost, state, path = heapq.heappop(queue)

        if state in visited:
            continue

        visited.add(state)

        if state == goal:
            return path

        for neighbor in get_neighbors(state):

            if neighbor not in visited:

                new_cost = cost + 1
                new_f = new_cost + heuristic(neighbor, goal)

                heapq.heappush(
                    queue,
```

```
                    (
                        new_f,
                        new_cost,
                        neighbor,
                        path + [neighbor]
                    )
                )

    return None


print("Enter initial state using 0 for blank:")
start = tuple(map(int, input().split()))

print("Enter goal state:")
goal = tuple(map(int, input().split()))

solution = a_star(start, goal)

if solution:
    print("\nSolution:")

    for state in solution:
        print(state[0:3])
        print(state[3:6])
        print(state[6:9])
        print()
else:
    print("No solution possible.")
```

Sample Input

```
Enter initial state using 0 for blank:
1 2 3 4 0 6 7 5 8

Enter goal state:
1 2 3 4 5 6 7 8 0
```

Sample Output

```
Solution:
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

Result

Thus, the 8-Puzzle Problem was successfully implemented using the A* Search algorithm with dynamic input.

EXPERIMENT 5: TRAVELLING SALESMAN PROBLEM

Aim

To implement the Travelling Salesman Problem using search techniques with dynamic input.

Algorithm

1.

Read the number of cities.

2.
3.

Read the distance matrix.

4.
5.

Read the starting city.

6.
7.

Generate all possible routes using permutations.

8.
9.

Calculate the total cost of each route.

10.
11.

Select the route having the minimum cost.

12.
13.

Return to the starting city.

14.
15.

Display the best route and minimum cost.

16.

Program

```
from itertools import permutations

n = int(input("Enter number of cities: "))

print("\nEnter the distance matrix:")

distance = []

for i in range(n):
    row = list(map(int, input(
        f"Enter distances for city {i}: "
    ).split()))

    distance.append(row)


start = int(input(
    f"\nEnter starting city (0 to {n - 1}): "
))


cities = []

for i in range(n):
    if i != start:
        cities.append(i)


min_cost = float('inf')
best_route = None


for perm in permutations(cities):

    route = [start] + list(perm) + [start]

    cost = 0

    for i in range(len(route) - 1):
        cost += distance[route[i]][route[i + 1]]

    if cost < min_cost:
        min_cost = cost
        best_route = route


print("\nBest Route:")
print(" -> ".join(map(str, best_route)))

print("Minimum Cost:", min_cost)
```

Sample Input

```
Enter number of cities: 4

Enter the distance matrix:
Enter distances for city 0: 0 10 15 20
Enter distances for city 1: 10 0 35 25
Enter distances for city 2: 15 35 0 30
Enter distances for city 3: 20 25 30 0

Enter starting city (0 to 3): 0
```

Sample Output

```
Best Route:
0 -> 1 -> 3 -> 2 -> 0

Minimum Cost: 80
```

Result

Thus, the Travelling Salesman Problem was successfully implemented using search techniques with dynamic input.